LAB EXERCISE 6

NAME : Yuvaraj R
REG NO : 26MCS1019

Q1. Given an **M × N** matrix of characters, find the length of the longest path in the matrix starting from a given character.

In alphabetical order, all characters in the longest path should be increasing and consecutive. We are allowed to search the string in all eight possible directions, i.e., North, West, South, East, North-East, North-West, South-East, and South-West.

Use depth-first search (DFS) to solve this problem.

For example, consider the following matrix of characters:

The length of the longest path with consecutive characters starting from character C is 6.

The longest path is:

Input Format
The first line of input is an integer **n** representing the row size.
The second line of input is an integer **m** representing the column size.
The following lines of input are the characters of the matrix.
The last line of the input is the character for which the length of the longest path is to be found.
Output Format
The output prints the length of the longest path with consecutive characters starting from the given character.

Refer to the sample output for formatting specifications.
Constraints
1 ≤ n,m ≤ 5
Sample Input Sample Output

```
5
5
d e h x b
a o g p e
d d c f d
e b e a s
c d y e n
c
```

```
The length of the longest path with consecutive characters starting from
character c is 6
```

Sample Input Sample Output

```
5
5
d e h x b
a o g p e
d d c f d
e b e a s
c d y e n
b
The length of the longest path with consecutive characters starting from
character b is 7
```

CODE :

```
#include <stdio.h>


int n, m;

char a[5][5];


/* 8 directions: N, NE, E, SE, S, SW, W, NW */

int row[] = {-1, -1, 0, 1, 1, 1, 0, -1};

int col[] = {0, 1, 1, 1, 0, -1, -1, -1};


int dfs(int r, int c)

{

  int i, nr, nc;

  int max = 1;


  for (i = 0; i < 8; i++)

  {

    nr = r + row[i];
```

```
        nc = c + col[i];

        if (nr >= 0 && nr < n && nc >= 0 && nc < m)
        {
            if (a[nr][nc] == a[r][c] + 1)
            {
                int length = 1 + dfs(nr, nc);

                if (length > max)
                    max = length;
            }
        }
    }

    return max;
}

int main()
{
    int i, j;
    char ch;
    int answer = 0;

    printf("Enter number of rows: ");
    scanf("%d", &n);
```

```
printf("Enter number of columns: ");
scanf("%d", &m);

printf("Enter the characters of the matrix:\n");

for (i = 0; i < n; i++)
{
   for (j = 0; j < m; j++)
   {
      scanf(" %c", &a[i][j]);
   }
}
printf("Enter the starting character: ");
scanf(" %c", &ch);

for (i = 0; i < n; i++)
{
   for (j = 0; j < m; j++)
   {
      if (a[i][j] == ch)
      {
         int length = dfs(i, j);

         if (length > answer)
```

```
            answer = length;
        }
      }
    }
    printf("\nThe length of the longest path with consecutive characters ");
    printf("starting from character %c is %d\n", ch, answer);
    return 0;
}
```

OUTPUT:

```
PS A:\VIT Chennai\DSAA_01\DSA - Lab\Lab-06> & "A:\VIT Chennai\DSAA_01\DSA - Lab\Lab-06\dfs.exe"
Enter number of rows: 3
Enter number of columns: 3
Enter the characters of the matrix:
a b c
d e f
g h i
Enter the starting character: e

The length of the longest path with consecutive characters starting from character e is 2
```

Q2. You are given a network of n computers, represented by vertices numbered from 0 to n-1. The computers are connected to each other through network cables, forming a communication network. Your task is to determine whether the given network is bipartite or not using the Breadth-first search algorithm.

A network is considered bipartite if it can be divided into two independent sets of computers such that no two computers within the same set are directly connected. In other words, the computers in each set can communicate with computers in the other set but not within their own set.

Input Format

The first line contains an integer n, representing the total number of computers in the network.
The next n lines contain n integers each, representing the adjacency matrix G.
Each integer G[i][j] is either 0 or 1, separated by a space.
The last line contains an integer src, representing the source computer.

Output Format

If the graph is bipartite, print "Yes, the given graph is Bipartite".
If the graph is not bipartite, print "No, the given graph is not Bipartite".

Refer to the sample output for the formatting specifications.
Constraints
1 ≤ n ≤ 6
The adjacency matrix will be a square matrix of size n x n.
Each cell in the matrix will contain either 0 or 1.
Sample Input Sample Output

```
3
0 1 1
1 0 0
1 0 0
0
Yes, the given graph is Bipartite
```

Sample Input Sample Output

```
5
0 1 1 0 0
1 0 0 1 0
1 0 0 0 1
0 1 0 0 1
0 0 1 1 0
3
No, the given graph is not Bipartite
```

CODE :

```
#include <stdio.h>



#define MAX 6



int main() {

  int n;

  int G[MAX][MAX];

  int color[MAX];

  int queue[MAX];

  int front = 0, rear = 0;
```

```
int src, i, j;
int bipartite = 1;

printf("Enter the number of computers: ");
scanf("%d", &n);

printf("Enter the adjacency matrix:\n");
for (i = 0; i < n; i++) {
   for (j = 0; j < n; j++) {
      scanf("%d", &G[i][j]);
   }
}

printf("Enter the source computer: ");
scanf("%d", &src);

// Initialize all vertices as uncolored
for (i = 0; i < n; i++) {
   color[i] = -1;
}

// Start BFS from source
color[src] = 0;
queue[rear++] = src;
```

```
while (front < rear && bipartite) {
   int u = queue[front++];

   for (j = 0; j < n; j++) {
      if (G[u][j] == 1) {

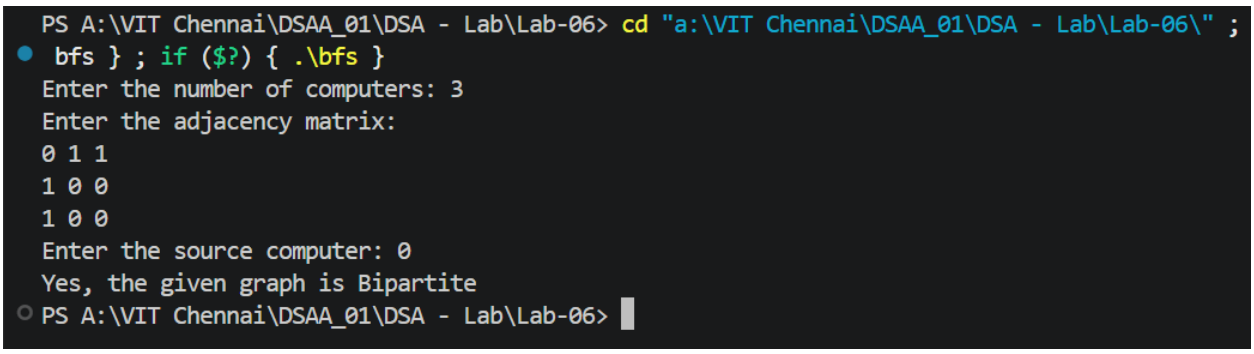
         // If vertex is not colored
         if (color[j] == -1) {
            color[j] = 1 - color[u];
            queue[rear++] = j;
         }

         // If adjacent vertices have same color
         else if (color[j] == color[u]) {
            bipartite = 0;
            break;
         }
      }
   }
}

if (bipartite)
   printf("Yes, the given graph is Bipartite");
else
   printf("No, the given graph is not Bipartite");
```

```
    return 0;
}
```

OUTPUT :



Q3. You are given a grid consisting of characters. Each character represents a different type of node. Implement a program to find the length of the longest path between two nodes of different types in the grid. The path can only move vertically or horizontally, not diagonally.

The grid is represented as a 2D array, with the number of rows and columns specified as input.

Each cell in the grid contains a single character. To move from one cell to another, you can only go to adjacent cells that share a side (up, down, left, or right). You cannot visit a cell more than once in a single path.

Write a function **findLongestPath** that takes the grid, the number of rows, and the number of columns as inputs and returns the length of the longest path between two nodes of different types.

Example

Input:
3 4
ABCD
EFGH
IJKL

Output:
12

Explanation:
The longest path in this case starts at the cell (0, 0) with the character 'A' and traverses the grid as follows:
(0, 0) -> (0, 1) -> (0, 2) -> (0, 3) -> (1, 3) -> (2, 3) -> (2, 2) -> (2, 1) -> (2, 0) -> (1, 0) -> (1, 1) -> (1, 2)
The path length is 12, which includes visiting all the cells in the grid.

Input Format

The first line of input consists of two space-separated integers: **M** and **N**, representing the dimensions of the grid.
The following **M** lines contain **N** characters each, representing the characters in the grid.

Output Format

The output prints an integer representing the length of the longest path.

Refer to the sample output for the formatting specifications.

Constraints

1 ≤ M, N ≤ 100
The characters in the grid will be uppercase alphabets (A-Z).
The grid will have at least two different types of nodes.
The grid will not contain any spaces or special characters.

Sample Input Sample Output

```
3 4
ABCD
EFGH
IJKL
12
```

Sample Input Sample Output

```
2 2
AB
CD
4
```

CODE:

```
#include <stdio.h>
#include <string.h>

#define MAX 100

int M, N;
char grid[MAX][MAX];
int visited[MAX][MAX];

int dr[] = {-1, 1, 0, 0};
int dc[] = {0, 0, -1, 1};

int isValid(int r, int c) {
```

```
    return r >= 0 && r < M &&
           c >= 0 && c < N &&
           !visited[r][c];
}

int dfs(int r, int c, char startType) {
    int i;
    int maxLength = 1;

    visited[r][c] = 1;

    for (i = 0; i < 4; i++) {
        int nr = r + dr[i];
        int nc = c + dc[i];

        if (isValid(nr, nc)) {
            int currentLength;

            /*
             * We can continue the path through any cell.
             * The final cell must have a different type
             * from the starting cell.
             */
            currentLength = dfs(nr, nc, startType) + 1;

            if (currentLength > maxLength)
                maxLength = currentLength;
        }
    }

    visited[r][c] = 0;

    return maxLength;
}

int findLongestPath() {
    int i, j;
    int longest = 0;

    for (i = 0; i < M; i++) {
        for (j = 0; j < N; j++) {
            int length;
```

```
            memset(visited, 0, sizeof(visited));

            length = dfs(i, j, grid[i][j]);

            if (length > longest)
                longest = length;
        }
    }

    return longest;
}

int main() {
    int i;

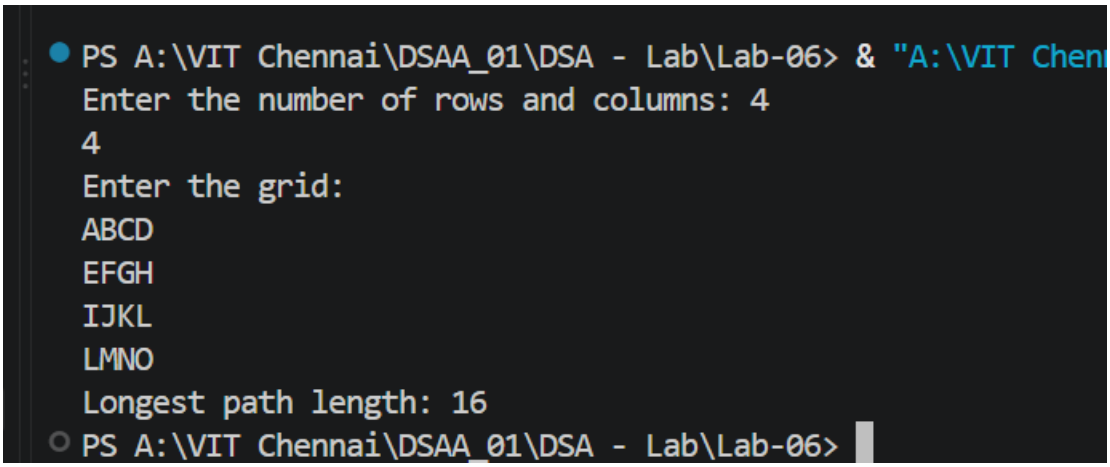
    printf("Enter the number of rows and columns: ");
    scanf("%d %d", &M, &N);

    printf("Enter the grid:\n");
    for (i = 0; i < M; i++) {
        scanf("%s", grid[i]);
    }

    printf("Longest path length: %d\n", findLongestPath());

    return 0;
}
```

OUTPUT:



Q4. Implement a program to find the shortest path in a weighted graph, given a source vertex and a destination vertex, using DFS and Dijkstra's algorithm.

The program should prompt the user to enter the number of vertices and edges in the graph, followed by the edges and their respective weights.

Finally, the program should ask the user for the source and destination vertices and print the shortest path and distance between them.

Input Format

The first line of input contains an integer **n,** denoting the number of vertices in the graph.

The second line of input contains an integer **m,** denoting the number of edges in the graph.

The next **m** lines each contain three space-separated integers u, v, and w denoting an edge between vertices u and v with weight w.

The next line of input contains an integer **s,** denoting the source vertex.

The last line of input contains an integer **d,** denoting the destination vertex.

Output Format

If there is no path from the source vertex to the destination vertex, the output displays "No path found".

If there is a path from the source vertex to the destination vertex:

1. The first line displays "Shortest path: " followed by the shortest path between them in the format "source -> vertex1 -> vertex2 -> ... -> destination".
2. The second line displays "Shortest distance: " followed by the shortest distance.

Refer to the sample output for formatting specifications.

Constraints

$1 \le n \le 100$

$1 \le m \le 100$

$0 \le u, v < n$

$0 \le w \le 100$

$0 \le s, d < n$

Sample Input Sample Output

```
4
5
0 1 2
0 2 4
1 2 1
1 3 7
2 3 3
0
3
Shortest path: 0 -> 1 -> 2 -> 3
Shortest distance: 6
```

Sample Input Sample Output

```
5
7
0 1 2
0 2 6
0 3 1
1 2 3
1 4 5
2 3 2
3 4 1
0
4
Shortest path: 0 -> 3 -> 4
Shortest distance: 2
```

Sample Input Sample Output

```
3
2
0 1 5
1 2 3
0
2
Shortest path: 0 -> 1 -> 2
Shortest distance: 8
```

Sample Input Sample Output

```
5
4
0 1 2
0 3 3
1 3 4
0 2 5
0
4
No path found
```

Code :

```
#include <stdio.h>

#define MAX 100
#define INF 999999

int main() {
    int n, m;
    int graph[MAX][MAX];
```

```
int dist[MAX], visited[MAX], parent[MAX];
int i, j;

// Initialize graph
for (i = 0; i < MAX; i++) {
    for (j = 0; j < MAX; j++) {
        graph[i][j] = -1;
    }
}

printf("Enter the number of vertices: ");
scanf("%d", &n);

printf("Enter the number of edges: ");
scanf("%d", &m);

printf("Enter the edges (u v weight):\n");

for (i = 0; i < m; i++) {
    int u, v, w;

    scanf("%d %d %d", &u, &v, &w);

    graph[u][v] = w;
    graph[v][u] = w;   // Undirected graph
}

int s, d;

printf("Enter the source vertex: ");
scanf("%d", &s);

printf("Enter the destination vertex: ");
scanf("%d", &d);

// Initialize Dijkstra arrays
for (i = 0; i < n; i++) {
    dist[i] = INF;
    visited[i] = 0;
    parent[i] = -1;
}

// Distance from source to itself is 0
```

```
dist[s] = 0;

// Dijkstra's algorithm
for (i = 0; i < n; i++) {

    int u = -1;

    // Find the unvisited vertex with minimum distance
    for (j = 0; j < n; j++) {
        if (!visited[j] &&
            (u == -1 || dist[j] < dist[u])) {
            u = j;
        }
    }

    // No reachable vertex remains
    if (u == -1 || dist[u] == INF)
        break;

    visited[u] = 1;

    // Update distances of adjacent vertices
    for (j = 0; j < n; j++) {
        if (graph[u][j] != -1 && !visited[j]) {

            if (dist[u] + graph[u][j] < dist[j]) {
                dist[j] = dist[u] + graph[u][j];
                parent[j] = u;
            }
        }
    }
}

// Check if destination is unreachable
if (dist[d] == INF) {
    printf("No path found");
}
else {
    int path[MAX];
    int count = 0;
    int current = d;

    // Store path backwards
```

```
        while (current != -1) {
            path[count++] = current;
            current = parent[current];
        }

        printf("Shortest path: ");

        // Print path in correct order
        for (i = count - 1; i >= 0; i--) {
            printf("%d", path[i]);

            if (i != 0)
                printf(" -> ");
        }

        printf("\n");
        printf("Shortest distance: %d", dist[d]);
    }

    return 0;
}
```

Output:

```
PS A:\VIT Chennai\DSAA_01\DSA - Lab\Lab-06> & "A:\VI
Enter the number of vertices: 4
Enter the number of edges: 5
Enter the edges (u v weight):
0 1 2
0 2 4
1 2 1
1 3 7
2 3 3
Enter the source vertex: 0
Enter the destination vertex: 3
Shortest path: 0 -> 1 -> 2 -> 3
Shortest distance: 6
PS A:\VIT Chennai\DSAA_01\DSA - Lab\Lab-06>
```

Q5. Imagine you are working on a project that involves analyzing various interconnected data points, represented as nodes, with connections between them, represented as edges.

Your task is to find the minimal spanning tree (MST) of this network, which is a subset of edges that connects all nodes without forming any cycles and has the minimum possible total edge weight.

To achieve this, you need to write a program that uses Prim's Algorithm to find the MST of a given graph.

Input Format

The first line of input consists of an integer **V,** representing the number of nodes.
The following **V** lines contain **V** space-separated integers each, representing the weighted connections between nodes.
The weight between node i and node j is provided in the ith row and j^{th} column of the input.

Output Format

The output should print the edges and their corresponding weights in the minimal spanning tree (MST).

Refer to the sample output for the exact format.

Constraints

$1 \le V \le 10$
$0 \le weight[i][j] \le 50$
$weight[i][i] = 0$
$Max_V = 10$

Sample Input Sample Output

```
3
0 2 3
2 0 4
3 4 0
Edge   Weight
0 - 1    2
0 - 2    3
```

Sample Input Sample Output

```
5
0 2 0 6 0
2 0 3 8 5
0 3 0 0 7
6 8 0 0 9
0 5 7 9 0
Edge   Weight
0 - 1    2
1 - 2    3
0 - 3    6
1 - 4    5
```

Code :

```c
#include <stdio.h>

#define MAX 10
#define INF 9999

int main() {
    int V;
    int graph[MAX][MAX];
    int selected[MAX];
    int i, j;
    int edges = 0;

    printf("Enter the number of vertices: ");
    scanf("%d", &V);

    printf("Enter the weighted adjacency matrix:\n");

    for (i = 0; i < V; i++) {
        for (j = 0; j < V; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    // Initially, no vertex is selected
    for (i = 0; i < V; i++) {
        selected[i] = 0;
    }

    // Start from vertex 0
    selected[0] = 1;

    printf("Edge   Weight\n");

    while (edges < V - 1) {
        int min = INF;
        int x = -1;
        int y = -1;

        // Find the minimum weight edge
        // connecting selected and unselected vertices
        for (i = 0; i < V; i++) {
            if (selected[i]) {
```

```
                for (j = 0; j < V; j++) {
                    if (!selected[j] &&
                        graph[i][j] != 0 &&
                        graph[i][j] < min) {

                        min = graph[i][j];
                        x = i;
                        y = j;
                    }
                }
            }
        }

        // If no edge is found, graph is disconnected
        if (x == -1 || y == -1) {
            break;
        }

        printf("%d - %d    %d\n", x, y, min);

        selected[y] = 1;
        edges++;
    }

    return 0;
}
```

OUTPUT:

```
PS A:\VIT Chennai\DSAA_01\DSA - Lab\Lab-06> & "A:\VI
Enter the number of vertices: 3
Enter the weighted adjacency matrix:
0 2 3
2 0 4
3 4 0
Edge   Weight
0 - 1    2
0 - 2    3
PS A:\VIT Chennai\DSAA_01\DSA - Lab\Lab-06>
```

Q6. You are given a set of cities in a country and the cost of building railway track segments to connect them. Your task is to find the minimum cost required to build the

railway tracks and connect all the cities. The cost of each track segment is given, along with the source city and destination city to which it connects.

Write a program that takes the number of cities, the number of track segments, and the details of each track segment as input. Implement Kruskal's Algorithm to calculate the minimum cost of building the railway tracks.

Example

Input:
3
3
0 1 4
1 2 5
0 2 3

Output:
7

Explanation:
The input specifies 3 cities and 3 track segments with their corresponding weights.

The track segments are:

1. Track Segment 1: Connects city 0 and city 1, with a weight of 4.
2. Track Segment 2: Connects city 1 and city 2 with a weight of 5.
3. Track Segment 3: Connects city 0 and city 2 with a weight of 3.

The algorithm finds the minimum cost to build railway tracks connecting all cities. In this case, the minimum spanning tree includes track segment 3 (0-2 with weight 3) and track segment 1 (0-1 with weight 4). The total cost is 3 + 4 = 7.

Input Format
The first line of input consists of an integer **N,** representing the number of cities.
The second line consists of an integer **M,** representing the number of track segments.
For each **M** track segment, provide the source city (0-based index), destination city (0-based index), and the weight (cost) of constructing the track segment.

Output Format
If it is possible to connect all cities, print the minimum cost to build the railway tracks.
Otherwise, print "It is not possible to connect all cities."

Refer to the sample output for the formatting specifications.

Constraints
1 ≤ N, M ≤ 10

Sample Input Sample Output

3

3

```
0 1 4
1 2 5
0 2 3
7
```

Sample Input Sample Output

```
6
8
0 1 1
1 2 2
2 3 3
3 4 4
4 0 5
1 4 6
1 3 7
2 4 8
It is not possible to connect all cities.
```

Code:

```
#include <stdio.h>

#define MAX 10

struct Edge {
    int u;
    int v;
    int w;
};

int parent[MAX];

int find(int x) {
    if (parent[x] == x)
        return x;

    return find(parent[x]);
}

void unionSet(int a, int b) {
    int rootA = find(a);
    int rootB = find(b);

    parent[rootA] = rootB;
}
```

```
int main() {
    int N, M;
    struct Edge edges[MAX * MAX];

    int i, j;
    int count = 0;
    int totalCost = 0;

    printf("Enter the number of cities: ");
    scanf("%d", &N);

    printf("Enter the number of track segments: ");
    scanf("%d", &M);

    printf("Enter the track segments (source destination cost):\n");

    for (i = 0; i < M; i++) {
        scanf("%d %d %d",
              &edges[i].u,
              &edges[i].v,
              &edges[i].w);
    }

    // Initialize each city as its own set
    for (i = 0; i < N; i++) {
        parent[i] = i;
    }

    // Sort edges by weight
    for (i = 0; i < M - 1; i++) {
        for (j = 0; j < M - i - 1; j++) {

            if (edges[j].w > edges[j + 1].w) {
                struct Edge temp = edges[j];
                edges[j] = edges[j + 1];
                edges[j + 1] = temp;
            }
        }
    }

    // Kruskal's Algorithm
    for (i = 0; i < M; i++) {
```

```
        int u = edges[i].u;
        int v = edges[i].v;

        // Check whether adding this edge creates a cycle
        if (find(u) != find(v)) {

            unionSet(u, v);

            totalCost += edges[i].w;
            count++;

            // MST needs N-1 edges
            if (count == N - 1)
                break;
        }
    }

    // Check whether all cities are connected
    if (count == N - 1) {
        printf("%d", totalCost);
    }
    else {
        printf("It is not possible to connect all cities.");
    }

    return 0;
}
```

Output:

